

SAP S/4HANA Integration with Keboola

Integration Guide

Setting up, running and troubleshooting the three integration flows, the value mapping and the Keboola adapter on SAP Integration Suite.

Contents

Overview	4
1. Before you start	5
What to deploy	5
How Keboola signs in to your tenant	5
Which topology	6
Setting up, in order	7
2. Connecting to SAP S/4HANA	7
Authentication	8
Topologies	9
Behind an SAP Cloud Connector: what to set up, and what to check first	9
3. Which services your user can reach	10
Views	11
The address book: OData V4 services on S/4HANA Cloud	11
Diagnostics	12
When the list cannot be read	13
Which arrangement grants which API	14
4. The pull connector	14
How large an answer can be	15
Retry	16
5. The Keboola side	17
Four things to know about the component	18
Reading the result	18
6. The delivery flow	18
The schedule and the manual start	21
What is retried, and what is not	21
Sizes and run times	23
More than one table	24
Full loads	24
Incremental loading	24
Set a primary key	25
What the flow does about a missing table	25
7. When something goes wrong	26
Alerting	27
Where a 401 or 403 comes from	28
What the monitor holds	28
The dead-letter queue	29
Editions without JMS	29
8. Things worth knowing before they surprise you	29
9. Rotating credentials	31
10. The Keboola adapter	31

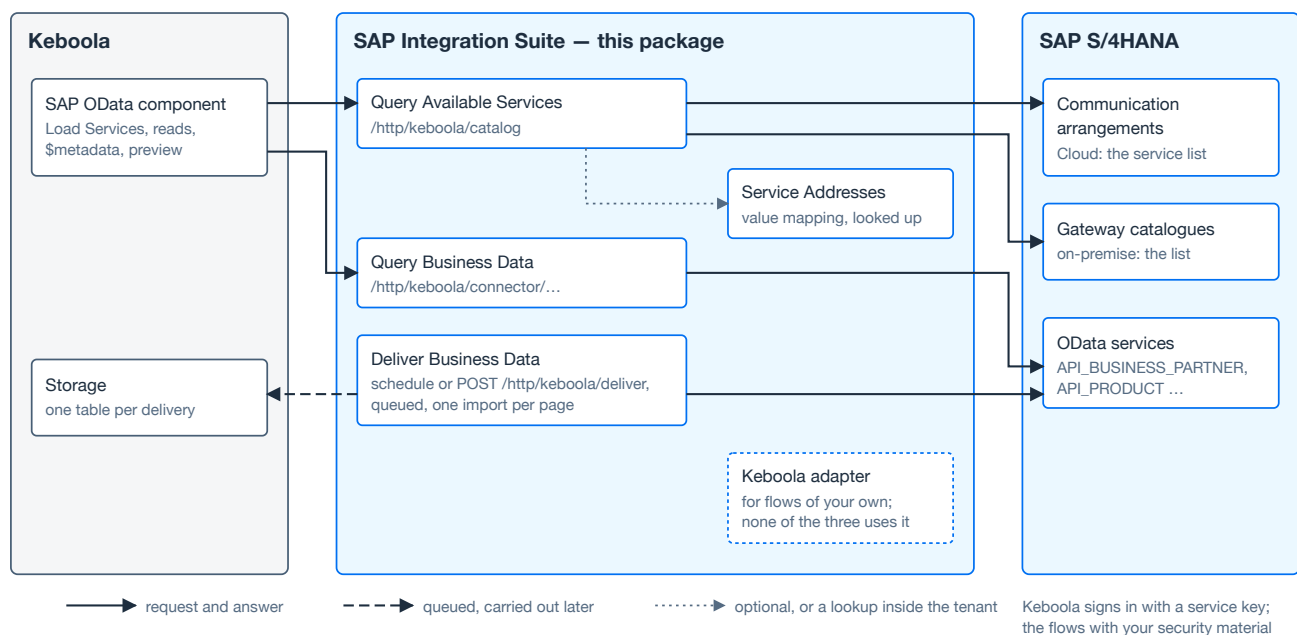
Settings	32
What your flow gets back	32
When it fails	33
Size	33
11. Support and responsibilities	34
12. Change log	34
Appendix A. Signing in to SAP S/4HANA, step by step	36
OAuth 2.0 Client Credentials	36
Client Certificate	36
Basic	36
Switching the method	37
When the sign-in does not work	37

Overview

This package connects an SAP S/4HANA system to the Keboola data platform. It contains three integration flows and one value mapping:

Artifact	What it does	When you use it
Query Available Services from SAP S4HANA	Lists the business data services your SAP user can reach, with their addresses	Once per connection, while setting it up — it fills the service list in Keboola
Query Business Data from SAP S4HANA to Keboola	Answers a request from Keboola by reading SAP and passing the answer back	Whenever Keboola asks for data
Deliver Business Data from SAP S4HANA to Keboola	Reads SAP on a schedule and writes the result into a Keboola table itself	When SAP should push, rather than Keboola pull
SAP S4HANA Service Addresses for Keboola	The address book the first flow reads: service name, address and title of the OData services SAP publishes	Deployed next to the first flow; you add rows for services it does not know

How they sit between the two systems:



The first two flows are the **pull** topology: Keboola asks, SAP answers. The third is the **push** topology: nothing outside your SAP tenant needs to be reachable, and the delivery survives a Keboola outage because it is queued.

Everything that differs between one installation and the next is on the Configure screen of each flow: the address of your SAP system, how the flow signs in to it, the role a caller must hold, and for the delivery flow its own address and schedule. Nothing in this guide asks you to open the flow editor. The one artifact you may edit is the value mapping, because that is what it is for (§3).

The package also carries the **Keboola adapter**, a receiver adapter for integration flows you build yourself. None of the flows needs it; §10 describes it.

1. Before you start

You need:

1. **An SAP Integration Suite tenant** with Cloud Integration enabled. The delivery flow also needs JMS message queues; editions without them can still use the two pull flows, and §7 describes what changes.
2. **A communication user on SAP S/4HANA** with a communication arrangement for each API you intend to read. §3 lists the arrangements for the common APIs.
3. **A Keboola project**, and — for the delivery flow — a Storage token with write access to the target bucket.

What to deploy

Artifact	Needed for	Deploy it?
Query Business Data from SAP S4HANA to Keboola	Every read Keboola's SAP OData component makes: data, \$metadata, \$count, the preview	Required
Query Available Services from SAP S4HANA	The service list the component offers on its configuration screen (<i>Load Services</i>)	Optional — a service path can also be typed by hand (§5)
SAP S4HANA Service Addresses for Keboola	The addresses and titles of OData V4 services on SAP S/4HANA Cloud, which the arrangements do not carry; the titles of OData V2 services	Optional; deploy it together with the catalogue flow on S/4HANA Cloud, otherwise a V4 service is listed only when one of four guessed addresses verifies (§3)
Deliver Business Data from SAP S4HANA to Keboola	The push topology: SAP writes into a Keboola table on a schedule	Optional
Keboola adapter	Integration flows of your own that write into Keboola (§10)	Optional

How Keboola signs in to your tenant

Keboola calls the flows with the client id and client secret of a service key. You create it in the BTP cockpit: a service instance of **SAP Process Integration Runtime**, plan `integration-flow`, created with the role the flows ask for, and a service key on that instance. The flows ship asking for `ESBMessaging.send`, a role SAP predefines in every tenant, so an instance created with `{"roles": ["ESBMessaging.send"]}` works with the package as delivered.

To let only Keboola in, use a role of your own. Every caller that holds `ESBMessaging.send` can call every flow in your tenant that asks for it. A role of your own narrows that to the callers you gave it to, and SAP's design guidelines recommend it:

1. **Monitor** → **Manage Security** → **User Roles** → **Add**. Name the role, for example `KeboolaIntegration.Send`. Letters, digits, underscore and period are allowed.
2. Create the service instance with that role — `{"roles": ["KeboolaIntegration.Send"]}` — and a service key on it. Give Keboola that key's client id and client secret.
3. On the Configure screen of each flow you use, set `SENDER_ROLE` to your role and deploy.

The platform checks the role before the flow runs: a caller without it is answered `403` and no run is written. A role name nobody holds is accepted at deploy, so a typo shows only as that `403`. Which role guards your tenant is your decision, which is why the package does not make it for you.

Reads need no token, writes do. The two pull flows have the platform's protection against cross-site request forgery switched on. Keboola only reads, and a read (`GET` , `HEAD`) is never checked, so nothing has to be set up for it. A `POST` , `PUT` or `DELETE` that carries no token is refused by the platform with `403` before the flow runs. The delivery flow's own endpoint, which is started with a `POST` , asks for no token.

Which topology

	Pull	Push
Who starts the transfer	Keboola	Your SAP tenant, on a schedule
Needs your tenant reachable from Keboola	yes	no
Survives Keboola being briefly unavailable	the job fails and Keboola retries	yes — the run waits in a queue
Row-level control from Keboola's side	yes, per job	no, the schedule decides

They are not exclusive. Most installations use the catalogue and pull flows to explore and develop, and the delivery flow for the tables that matter in production.

Setting up, in order

1. **The service key for Keboola**, as described above.
2. **The security material for SAP S/4HANA**. Name it `KEB00LA_S4` and there is nothing to enter in `S4_CREDENTIAL_ALIAS`, which ships with that name. Its type follows the sign-in method you choose in `S4_AUTH_METHOD` (§2, Appendix A): *OAuth2 Client Credentials* for the shipped method, *User Credentials* for Basic, a key pair in the keystore for a client certificate.
3. **For the delivery flow, the security material for Keboola**: *User Credentials* named `KEB00LA_STORAGE`, with any user name and the Storage token as the password.
4. **Deploy the value mapping** *SAP S/4HANA Service Addresses for Keboola* if you deploy the catalogue flow. It needs no configuration.
5. **Configure** each flow you use: the fields that name your systems ship empty, the sign-in method ships as *OAuth 2.0 Client Credentials*, the role as `ESBMessaging.send`, the delivery flow's address as `/keboola/deliver` and its schedule as every day at 02:00 UTC.
6. **Deploy**.

A flow that is deployed before it is configured says so. Its first call answers `500 CONFIG_ERROR` and names the setting, for example `S4_HOSTNAME` is not configured on this integration flow. A sign-in method that is misspelt on the Configure screen fails the deploy itself — the runtime reports the artifact as *Error* — and the copy that was running before keeps answering until you correct it.

2. Connecting to SAP S/4HANA

All three flows share the same connection settings.

Parameter	Meaning
<code>S4_HOSTNAME</code>	Base address of the S/4HANA system, including the protocol, for example <code>https://my123456-api.s4hana.cloud.sap</code> . Must start with https:// — except behind an SAP Cloud Connector, where it is the virtual host written with <code>http://</code> (below)
<code>S4_AUTH_METHOD</code>	How the flow signs in: <i>OAuth2 Client Credentials</i> (shipped), <i>Basic</i> or <i>Client Certificate</i> , chosen from the list the screen offers
<code>S4_CREDENTIAL_ALIAS</code>	Name of the security material that holds the sign-in details. Ships as <code>KEB00LA_S4</code> . Not used with a client certificate
<code>S4_PRIVATE_KEY_ALIAS</code>	Alias of the key pair in the tenant's keystore, for <i>Client Certificate</i> only. Ships empty
<code>HTTP_TIMEOUT_MS</code>	How long one call may take, in milliseconds. Ships as <code>50000</code> on the pull connector, <code>10000</code> on the catalogue and <code>120000</code> on the delivery flow
<code>S4_PROXY_TYPE</code>	<code>default</code> for a system on the internet, <code>sapcc</code> for one behind an SAP Cloud Connector
<code>S4_CC_LOCATION_ID</code>	Cloud Connector location, only if you run more than one
<code>S4_SAP_CLIENT</code>	Client number on a multi-client on-premise system; sent as <code>sap-client</code> with every call. Leave empty on S/4HANA Cloud

The `https://` rule has one exception, and it is SAP's. With `S4_PROXY_TYPE=sapcc` the address is the *virtual host* you defined in the Cloud Connector, and SAP requires it to be written with `http://` — for example `http://s4hana.virtual:44300`. The platform answers an `https://` address on that route with 405 HTTPS proxying is not supported. Use HTTP instead. Nothing travels unencrypted: the tenant talks to the Cloud Connector through its encrypted tunnel, and the protocol of the last hop to SAP is the one you choose in the Cloud Connector's system mapping. With any other proxy type the rule stands: the pull connector refuses every request against an `http://` address with 500 CONFIG_ERROR naming the parameter; the catalogue does the same, and also refuses an address written without its protocol; the delivery flow refuses to start a run — and its manual endpoint answers the same 500 CONFIG_ERROR — for an `http://` value in `S4_HOSTNAME`, `KEB00LA_STACK_URL` or `KEB00LA_IMPORTER_URL`. The alternative would be sending your SAP credential, your Storage token and every row of business data across the network in clear text.

Authentication

The sign-in method is a Configure value. `S4_AUTH_METHOD` ships as **OAuth 2.0 Client Credentials**; to use another method choose it from the list on the Configure screen, name the matching security material in `S4_CREDENTIAL_ALIAS` — or, for a certificate, the key pair in `S4_PRIVATE_KEY_ALIAS` — and deploy. The three fields belong together on every connection to SAP in the package; the catalogue's four connections and the other two flows' one each all take the same three values. One build of the package serves every method: nothing is edited.

Your system	Use	Why
S/4HANA Cloud, public edition	OAuth 2.0 Client Credentials (the shipped setting), Basic, or Client Certificate	The communication arrangement's inbound communication offers a user name and password, a certificate mapped to the communication user, or — for the scenarios that support it — OAuth 2.0 client credentials with a client id, a secret and the system's token endpoint
On-premise or private edition, reachable directly	OAuth 2.0 Client Credentials, Basic, or Client Certificate	OAuth 2.0 client credentials are issued by the ABAP OAuth server; a certificate is mapped to the communication user on the SAP system
On-premise behind an SAP Cloud Connector (<code>S4_PROXY_TYPE=sapcc</code>)	Basic	SAP's HTTP adapter does not allow a client certificate on that route, and a token request to an on-premise address fails there. The credential still never leaves the encrypted tunnel. Applies to all three flows
Either, to get started	Basic	Works everywhere over TLS; SAP's design guidelines flag it as not the strongest choice

The catalogue under OAuth 2.0 on an on-premise system. An ABAP OAuth client sees only the OData services assigned to it as scopes. The catalogue reads two more services than the data flows do — the Gateway catalogues `/sap/opu/odata/IWFND/CATALOGSERVICE;v=2` and `/sap/opu/odata4/iwfnd/config/default/iwfnd/catalog/0002` (§3) — so with OAuth 2.0 Client Credentials against an on-premise system those two need to be in the client's scope as well, or the catalogue answers 502 UPSTREAM_ERROR naming the three statuses it got while the pull connector still reads the business data. On S/4HANA Cloud the communication arrangement grants everything the catalogue needs.

When the security material does not fit. A material that is missing, or of another type than the sign-in method needs, is answered by the two pull flows with `500 CONFIG_ERROR` and a sentence that names `S4_CREDENTIAL_ALIAS` and `S4_AUTH_METHOD`, followed by the platform's own words; the run carries the place inside the platform where the sign-in failed as the field `ErrorLocation`. In the delivery flow the same fault sends the run to `RETRY`, where it waits until the material is corrected (§6).

Appendix A has the steps for each method, click by click.

Topologies

- **S/4HANA Cloud, public edition.** Reachable on the internet; set `S4_PROXY_TYPE=default`. The Gateway service catalogues are not available on this edition; the catalogue flow reads the communication arrangements instead (§3).
- **On-premise or private edition, exposed.** Same as above, pointing at your gateway host.
- **On-premise behind an SAP Cloud Connector.** Set `S4_PROXY_TYPE=sapcc`, `S4_HOSTNAME` to `http://` plus the virtual host and port, and `S4_CC_LOCATION_ID` if you run more than one Cloud Connector. Everything else stays as it is.

Behind an SAP Cloud Connector: what to set up, and what to check first

All three flows were tested through an SAP Cloud Connector, version 2.17.2, and behave as they do over the internet: the same reads, the same catalogue and the same delivery.

In the Cloud Connector, under *Cloud To On-Premise* › *Access Control*:

Setting	Value
Back-end Type	ABAP System
Protocol	the one your SAP port speaks — HTTPS for an HTTPS port
Internal Host and Port	your SAP system, by the name its certificate is issued to
Virtual Host and Port	a name of your choice; this is what goes into <code>S4_HOSTNAME</code>
Principal Type	None
Resource	<code>/sap/opu/odata</code> , <i>Path And All Sub-Paths</i> — it covers <code>/sap/opu/odata4</code> as well, the rule is "starts with"

Three things look fine and are not. Check them in this order:

1. **The mapping says *Not Reachable at once***. The protocol does not match the port: plain HTTP against an HTTPS port gets no answer.
2. **The mapping says *Not Reachable after a few seconds***, and *Configuration › On Premise › Backend Trust Store* reads "No trusted backends are configured". The Cloud Connector trusts no system over HTTPS until a certificate is on its allowlist. Add the certificate of the authority that signed your SAP system's certificate, or that certificate itself.
3. **The Cloud Connector says *Connected*, and every call answers *503 Timed out waiting for tunnel to open after ten seconds***. The Cloud Connector's Java does not know the authority that signs the tunnel host of your tenant's landscape. Pairing goes to one SAP host, the data tunnel to another (`connectivitytunnel.cf.<landscape>.hana.ondemand.com`), and on the newer landscapes that one is signed under *DigiCert TLS RSA4096 Root G5*, which a Java from before 2021 does not carry. The Cloud Connector's log (`log/scc_core.trc`) then says `unable to find valid certification path to requested target`. Update the Java the Cloud Connector runs on, or add that root to its trust list (`cacerts`), and restart the Cloud Connector.

SAP's own test tells you whether the tenant reaches the Cloud Connector, without any flow: **Monitor › Manage Security › Connectivity Tests › Cloud Connector**, with your Location ID or an empty one.

The addresses inside SAP's answers (`__metadata` , `links`) name the virtual host on this route. The flows do not rewrite them; §4 says what that means for the pull connector.

3. Which services your user can reach

The catalogue flow answers on `https://<your tenant>/http/keboola/catalog` and returns the services your communication user can reach, each with its address. Every read of SAP it makes goes through the flow's own connections to SAP — the ones that carry your proxy type, Cloud Connector location, sign-in method and certificate — and is visible in the message monitor and in a trace. It reads from one of two sources, chosen by `CATALOG_SOURCE` :

1. **Communication arrangements** — the authoritative list on S/4HANA Cloud. Needs the arrangement for `SAP_COM_0A07` (communication arrangement read). The flow reads the arrangements, their inbound services and their inbound users, page by page, and keeps the services of the arrangements that name your sign-in user (the user name of the security material, or the client id of an OAuth 2.0 material). When the user is not named by any arrangement, or the method carries no user name, every arrangement's services are listed and the diagnostics say so.
2. **The Gateway service catalogues** — available on on-premise and private-edition systems. Both the OData V2 catalogue (`/sap/opu/odata/IWFND/CATALOGSERVICE;v=2`) and the OData V4 catalogue (`/sap/opu/odata4/iwfnd/config/default/iwfnd/catalog/0002`) are read and merged, so V2 services and V4 service groups both appear, each with the address SAP itself lists for it.

`CATALOG_SOURCE` ships as `auto` : the arrangements first, and the Gateway catalogues when the arrangement API answers that it does not exist, as an on-premise system does. `arrangements` and `gateway` force one source. The response header `X-Keboola-Catalog-Source` says which one answered. Behind an SAP Cloud Connector the flow reads the two Gateway catalogues through the tunnel like any other call.

On S/4HANA Cloud the Gateway catalogues answer 403 and always will: no communication scenario grants them. That is not a misconfiguration and nothing you set will change it; the arrangements source is the one that works there.

An on-premise catalogue lists what exists, not what your user may read. The flow makes no authorization check of its own, so a service in the list can still answer 403 to your user when it is read, and an OData V4 service can answer 500 with `Service is not registered` until it is activated on the Gateway (transaction `/IWFND/V4_ADMIN`). Keboola's component reports both when the service is read; the list itself is not the place to find out. On S/4HANA Cloud the list is curated already: an arrangement grants what it lists.

There is no cache: every call reads SAP afresh, and every call is one message in the monitor whatever the number of reads it makes.

Views

On an on-premise system the Gateway catalogues list everything — on a typical S/4HANA that is ten thousand entries, most of them the backends of Fiori apps and raw CDS views. The flow answers with one of three views, chosen with `?include=`:

View	What it keeps	Typical size
interfaces (default)	Services SAP classifies as WEB_API, services you built yourself, anything named API_*, and OData V4 services from the released repository (s rvd_a2x)	a few hundred
extended	Everything SAP does not classify as a UI service; of OData V4, the released repository and the services you built yourself	under a thousand
all	Everything the catalogues list	everything

The view is the `CATALOG_VIEW` parameter (`interfaces` unless you change it); `?include=` on the address overrides it for one call, and a value that is none of the three is ignored. The views nest, and the response header `X-Keboola-Catalog-View` says which one answered and what it hid (`interfaces;shown=204;hidden=9809`). The list is sorted with `API_*` services first, then alphabetically, by technical name — a service your system registered as `ZAPI_BUSINESS_PARTNER_0001` sorts as `API_BUSINESS_PARTNER` . Deprecated services stay and say so in their description. On S/4HANA Cloud the views are identical.

The address book: OData V4 services on S/4HANA Cloud

A communication arrangement names an OData V4 service by its service group, `API_PRODUCT` for instance, and carries no address; the address of a V2 service follows from its name (`/sap/opu/odata/sap/API_BUSINESS_PARTNER`), the address of a V4 service does not (`/sap/opu/odata4/sap/api_product/srvd_a2x/sap/product/0001`). The package therefore ships a value mapping, **SAP S4HANA Service Addresses for Keboola**, as its address book: for each service name, its address and its human title. It holds the OData services of SAP's published catalogue of released APIs for SAP S/4HANA Cloud as of September 2026 — 584 names, 395 addresses, 584 titles, every OData V4 service in the catalogue and the OData V2 services whose address does not follow from their name. A service name that exists in several versions is listed under its plain name for the lowest version and under `<name>_<version>` , such as `API_PRODUCT_0002` , for each further one.

How the catalogue uses it is `CATALOG_RESOLVE_V4` :

Value	What happens to a V4 service of an arrangement
lookup+verify (shipped)	The address comes from the value mapping. A name the mapping does not know is tried at up to four addresses built from its name — one small request each, to the service's own document, never to data — and listed at the first that answers. At most <code>CATALOG_VERIFY_LIMIT</code> such requests per call to the catalogue, 20 as shipped, 0 to 200; each takes about a tenth of a second on S/4HANA Cloud
lookup	The value mapping only; a name it does not know is left out
off	No OData V4 service of an arrangement is listed

Titles come from the mapping too, for V2 and V4 alike (`API_BUSINESS_PARTNER` – Business Partner (A2X) (OData V2) in Keboola's list); a name without a title is shown as `via <communication scenario>`.

What is left out is named, never silently dropped. The response header `X-Keboola-Catalog-Unlisted` counts the V4 services without a known address, the run in the monitor carries their names in the field `CatalogUnlistedNames`, and the diagnostics (below) list them. The remedy is a row in the value mapping.

Adding a service to the address book — a V4 service the mapping does not know, a service of your own (`Z*`, `Y*`) on S/4HANA Cloud, or a better title:

1. **Design** → **the package** → **SAP S4HANA Service Addresses for Keboola** → **Edit**. The mapping has one agency, `Keboola`, on both sides.
2. Add a pair `SERVICE_NAME` → `SERVICE_PATH`: the service name as the arrangement shows it, without the `_G4BA` suffix and in upper case (`API_PRODUCT`), and the address without the host (`/sap/opu/odata4/sap/api_product/srvd_a2x/sap/product/0001`).
3. Add a second pair `SERVICE_NAME` → `SERVICE_TITLE` with the same name and the title you want to see in Keboola. Every title and every address in the mapping must be unique — the runtime refuses to deploy a mapping in which two names share a title or an address, and it says nothing more than that the deploy failed.
4. **Save and Deploy the value mapping.** The running catalogue flow reads the new row within a few seconds; it is not redeployed.

The address of an inbound service is shown in the *Communication Arrangements* app of your S/4HANA Cloud system: open the arrangement, and under *Inbound Services* each service lists its service URL, of which the part after the host is the address. A V2 service of your own at `/sap/opu/odata/sap/<name>` needs no row: that address follows from the name.

An edited value mapping is yours: a later version of the package brings a new mapping, and the rows you added have to be added again. Keep a note of them.

Diagnostics

Every answer carries a run identifier — the platform's own message id — as `messageId` in error bodies and as the `RunId` property of the run in the message monitor. With `CATALOG_DIAGNOSTICS=true`, `?debug=1` on the address adds the diagnostics to the answer under `d.diagnostics` (the component reads `d.results` only): which source answered, every read of SAP with its status and duration, the services left out and why. The

parameter ships as `false` and should stay so unless support asks for it — the diagnostics name the system's answers, and every caller that holds the endpoint's role could read them. Whatever the setting, the run in the monitor always carries the same notes as the log property `CatalogDiagnostics`, on the run's *Properties* tab.

OData V2 and V4 services are listed together, each row's description ending in `(OData V2)` or `(OData V4)`, because the component's dropdown shows *Title - Description* and its *OData Version* field offers `v2` and `v4` by hand next to `auto`. The component does not tell the catalogue which one the user picked, so the list is never filtered by version on its own. `?odata=v2` or `?odata=v4` keeps one kind, for a support call.

On a multi-client on-premise system set `S4_SAP_CLIENT`; it is sent as `sap-client` with every call. Leave it empty on S/4HANA Cloud.

When the list cannot be read

A list that could not be read is an error, not an empty list. The flow answers:

Answer	When
502 UPSTREAM_ERROR	SAP refused the sign-in (401); SAP answered a catalogue request with a server error; on S/4HANA Cloud, the communication user has no arrangement for SAP_COM_0A07 (the message names the scenario); with CATALOG_SOURCE=arrangements, the system has no arrangement API
500 CONFIG_ERROR	S4_HOSTNAME names a sign-in page, a web front end or a redirecting address instead of the API address of the system: the arrangement API and both Gateway catalogues answered 404, a redirect or a page that is not JSON. Nothing is listed; the message says what to enter
502 UPSTREAM_UNREACHABLE	The system could not be reached: no such host, connection refused, the Cloud Connector not connected
504 UPSTREAM_TIMEOUT	A call took longer than HTTP_TIMEOUT_MS, or the arrangement pages did not finish within the flow's own deadline of 35 seconds
200 with the status CATALOG_EMPTY	The source answered and listed nothing readable — almost always a communication user not yet assigned to an arrangement

One call to the catalogue finishes within 50 seconds in every case: each read of SAP is bounded by `HTTP_TIMEOUT_MS` (shipped `10000`), no read is retried, no arrangement page or verification request starts after 35 seconds, and the calls run one after another. Keboola's component waits 60 seconds for the list.

Which arrangement grants which API

Each API needs its own communication arrangement. The common ones:

Data	OData service	Communication scenario
Business partners, customers and suppliers	API_BUSINESS_PARTNER	SAP_COM_0008 Business Partner, Customer and Supplier Integration
Products	API_PRODUCT_SRV	SAP_COM_0009 Product Integration
Sales orders	API_SALES_ORDER_SRV	SAP_COM_0109 Sales Order Integration
Purchase orders	API_PURCHASEORDER_PROCESS_SRV (deprecated by SAP)	SAP_COM_0053 Purchase Order Integration
Supplier invoices	API_SUPPLIERINVOICE_PROCESS_SRV	SAP_COM_0057 Supplier Invoice Integration
G/L accounts	API_GLACCOUNTINCHARTOFACCOUNTS_SRV	SAP_COM_0241 Finance - Cloud Consolidation Data Collection Integration, or SAP_COM_0087 SAP Analytics Cloud for Planning Integration
Journal entry items	API_JOURNALENTTRYITEMBASIC_SRV	SAP_COM_0241 or SAP_COM_0087, as above
G/L account line items	API_GLACCOUNTLINEITEM	SAP_COM_0303 Finance - Accounting Analytics Integration

Customers and suppliers are read through the business partner service, so they need no arrangement of their own. SAP_COM_0002 , Finance - Posting Integration, is for posting journal entries and does not grant reading them.

Sources of the table: the communication arrangements of an SAP S/4HANA Cloud Public Edition tenant of release 2608, read through the arrangement API in September 2026, which give the scenario id and the inbound services of every arrangement that exists there; the SAP Help pages *Communication Scenarios* (sales order management) and *APIs for General Ledger Accounting* for the same release; and the service document of each service on that tenant, which confirmed the spelling of the eight addresses. The app *Display Communication Scenarios* in your own system is the authority for your release.

4. The pull connector

Keboola calls `https://<your tenant>/http/keboola/connector/<the full OData path>`. The path after `/connector/` is passed to SAP unchanged, so it must be the complete service path — `/sap/opu/odata/sap/API_BUSINESS_PARTNER/A_BusinessPartner`, not just the service name.

It answers `GET` and `HEAD`. Anything else is refused: by the platform with `403` when the request carries no CSRF token (§1), and by the flow with `405` and an `Allow` header when it does. The connector is a read path, and a write verb would reach SAP with an empty body and fail confusingly at the far end.

Only the OData roots are forwarded. `CONNECTOR_PATH_PREFIXES` lists the path prefixes the connector passes on to SAP; it ships as `/sap/opu/odata/`, `/sap/opu/odata4/`. A request for a path outside them — `/sap/bc/ping`, say — is answered `403 PATH_NOT_ALLOWED` with a body that names the parameter, the path and the run, and never reaches SAP. Add a prefix for a service that lives elsewhere, or leave the parameter empty to forward every path, in which case only the communication user's authorizations limit what can be read. The path is normalised first, so `..` segments cannot step around a prefix.

Some paths never reach the flow. The platform rejects a path that contains an encoded slash (`%2F`), an encoded backslash (`%5C`), an encoded percent sign (`%25`), a line break, a double slash or `/../` with `400`, and logs no run. An OData key that holds such a character cannot be addressed through the connector.

The flow makes one call to SAP per request and passes SAP's status and body back untouched. Five request headers travel from Keboola to SAP: `Accept`, `Accept-Language`, `Prefer`, `If-None-Match` and `If-Modified-Since` — plus the integration key's header once SAP issues one (§8); everything else stops at the connector, including the tenant's own message identifiers. Of SAP's response headers, the ones an OData client uses travel back: `Content-Type`, `Content-Length`, `Content-Encoding`, `Content-Language`, `ETag`, `Last-Modified`, `Cache-Control`, `DataServiceVersion`, `OData-Version`, `sap-message`, `sap-messagescount`, `Location` and `Retry-After`. On a multi-client on-premise system set `S4_SAP_CLIENT`; the connector then adds `sap-client` to every query that does not already carry one.

What can be read is decided in four places: the prefixes above, the authorizations of the user in your security material, the resources you released in the Cloud Connector, and the role that guards the endpoint in your tenant (§1).

Addresses inside SAP's answers are not rewritten. SAP's answers carry addresses of their own: of each record, of the next page when SAP pages an answer itself, and of the place a redirect leads to. They name your SAP system, not the connector, and the connector passes them on as they are. A caller that follows one leaves the connector, and Keboola's component follows both redirects and next-page addresses. So give Keboola the service path that SAP answers directly: SAP redirects the address of a service that is written without its final slash. The sign of a caller that left the connector is a Keboola job that fails with a `401` from the SAP system, or with a host that cannot be found when the system is behind an SAP Cloud Connector. §5 says how to keep the component inside the connector.

How large an answer can be

SAP's Gateway caps a page at a number of records of its own, whatever `$top` the caller asks for, and sends the rest as a next-page link: on the on-premise system this package was tested against the cap is 5000 records, so a request for 20000 answers 5000 and a `__next` link. The largest answer the connector was tested with is a page of 5000 wide purchase order items, 22.6 MB of JSON, answered in 5.5 seconds; a page of 5000 narrow records (four fields) is 2 MB in about a second, and the same page in SAP's XML form twice that. The connector holds an answer in memory once and does not stream it, so the cap of your Gateway bounds what one request costs the tenant; nothing in the connector limits the size.

Retry

The connector does not retry; Keboola does. Keboola's SAP OData component retries an answer of 500 , 502 , 503 or 504 three times with short waits, and both the number of retries and its timeout are settings in the component's connection options. The connector's part is to tell the truth:

What happened	What Keboola receives	Keboola retries?
SAP answered	SAP's own status and body	5xx yes; 4xx no — a 403 will still be a 403
SAP could not be reached — no DNS, refused connection, certificate failure	502 UPSTREAM_UNREACHABLE with the cause in the body	yes
SAP did not answer within HTTP_TIMEOUT_MS	504 UPSTREAM_TIMEOUT	yes
The path is outside CONNECTOR_PATH_PREFIXES	403 PATH_NOT_ALLOWED naming the parameter and the path	no — add the prefix, or correct the service path
This flow is misconfigured	500 CONFIG_ERROR naming the parameter	yes, uselessly — fix the parameter
The security material named in S4_CREDENTIAL_ALIAS is missing, or of another type than S4_AUTH_METHOD needs	500 CONFIG_ERROR naming both parameters and the platform's own words	yes, uselessly — correct the security material
The platform itself failed inside the connector, which does not happen in normal operation	500 CONNECTOR_ERROR naming the place inside the platform	yes; the run's ErrorLocation field is what to send to support

Keep HTTP_TIMEOUT_MS (shipped 50000) below the component's own timeout — 60 seconds unless changed — so that a slow SAP system produces a 504 Keboola can retry rather than a dropped connection. There is one retry layer, so one read costs at most four calls to SAP under the component's defaults, every one of them a separately billed message.

Every error body carries messageId : the run's Message GUID in **Monitor** → **Message Processing Logs**, so a failed Keboola job is matched to its run without any header from Keboola.

5. The Keboola side

Keboola reads through this package with its **SAP OData** component (`keboola.ex-sap-odata`). This is the configuration that works with the package as delivered; the field names are the component's.

Field	Value
Base URL	<code>https://</code> plus the runtime host of your tenant — the host named in the url of the service key from §1 — with no path after it
Catalog Path	<code>http/keboola/catalog</code> — the catalogue flow. Leave it empty if you do not deploy that flow
Authentication Type	<i>Basic</i> , with the service key's client id as the user name and its client secret as the password. The runtime accepts the key's credentials this way
Request Timeout	60 seconds, the component's default — above the connector's HTTP_TIMEOUT_MS of 50 seconds and the catalogue's 50-second bound, so the component receives a status it can act on rather than a dropped line
Max Retries	3, the default. Every retry is a billed message (§8)
Default Page Size	1000, the default, is a sound choice; never set it above the page cap of your SAP Gateway (§4), for the reason given below
Per entity: Service Path	<code>http/keboola/connector/sap/opu/odata/sap/API_BUSINESS_PARTNER</code> — the connector's address followed by the full service path. <i>Load Services</i> fills it in from the catalogue: each entry's value is such a path, its label the service's name and title
Per entity: OData Version	v2 or v4, as the catalogue's description says — set it explicitly rather than auto, for the reason given below
Per entity: entity set, fields, filter, fetch type, incremental field, load type, primary key	As for any OData source

Load Services is the only thing the catalogue flow is for. Without it, type the service path by hand in the form above; the connector reads any service the communication user may reach within

`CONNECTOR_PATH_PREFIXES` .

Four things to know about the component

These are behaviours of the component at the time of writing; its own release notes say when they change.

1. **Next-page links are followed as they are.** When SAP's Gateway caps a page below the page size you asked for, it answers with a `__next` (V2) or `@odata.nextLink` (V4) address that names your SAP system. The component follows it, leaving the connector: the job then fails with `401` from SAP, or with an unknown host behind a Cloud Connector. Keep the page size at or below your Gateway's cap — 5000 on the system this package was tested against — so that SAP never has to send a link. Find your cap by reading one entity set with a large `$top` through the connector and looking for `__next` in the answer.
2. **The connector's error text does not reach the job log.** A failed request is shown with its status and address only. The connector's `code`, `message` and `messageId` are in the run in **Monitor** → **Message Processing Logs**: find it by time, or in **Monitor** → **Message Search** by the path that was read, which is the run's *Application Message ID*.
3. **Test Connection does not touch the connector.** It reports success when the base URL and the catalogue answer; it proves the service key and, with a catalogue path, the catalogue flow, not that a service path is right. Use *Preview Data* or a small job for that.
4. **Zero rows instead of an error.** With *OData Version* on `auto` the component reads the service's `$metadata` to tell V2 from V4; when that read fails for any reason other than a sign-in refusal — a `502` or `504` from the connector while SAP is briefly unavailable, for example — it assumes V2. A V4 service then delivers no rows and no error. Set the version explicitly on every entity.

Two more, smaller: *Load Services* calls the catalogue twice per click, so expect two runs in the monitor for one click; and a *Load Services* failure that leaves no run in the monitor never reached your tenant — Keboola's own service answered, and a second try is the remedy.

Reading the result

A table written by the component carries the columns of the service, typed as the component decides from `$metadata`. A job that fails shows the component's own error; a `401` or `403` from the connector has three possible origins, told apart in §7.

6. The delivery flow

A run is started by the schedule or by a `POST` to `https://<your tenant>/http/<DELIVERY_ADDRESS>`, `/http/keboola/deliver` as shipped (any other verb answers `405`), put on a queue, and carried out by a worker that reads SAP page by page and writes each page into Keboola.

The queue is the point. Because a run is a queued message, a failure that may pass — a server error, a dropped line, a timeout — puts the run back on the queue and the platform retries it with exponential backoff, at the two intervals you configure; while it waits it is a failed message you can see. A failure that will not

change — not authorized, a wrong path, a wrong table — ends the run with the platform status **Escalated** and a custom status you can filter on, and is not retried: the next scheduled run is the retry, once the cause is fixed. Same rule as the pull connector, where Keboola retries 5xx and never a 4xx (§4).

Parameter	Default	Meaning
SERVICE_PATH	/sap/opu/odata/sap/API_BUSINESS_PARTNER	Path of the service to read, without the host
ENTITY_SET	A_BusinessPartner	The collection to read from that service. A view with parameters is written whole, as in <code>Name(P_x=...)/Results</code>
ODATA_SELECT	empty	Fields to read, separated by commas; empty or * reads every field. Naming them fixes the column order and stops a later SAP release adding columns silently. Must include the columns of PRIMARY_KEY. A structured field cannot be named
ODATA_FILTER	empty	Extra condition, in SAP's query language
LOAD_MODE	full	full replaces the table's contents on every run; incremental updates it by primary key
DELTA_FIELD	LastChangeDate,CreationDate	Field that says when a record changed; several fields are separated by commas
DELTA_FIELD_TYPE	datetime	How those fields are typed: datetime, datetimeoffset or date
DELTA_PRECISION	day	day if SAP records only the date of a change, second if it records the time, in universal time
DELTA_OVERLAP_MINUTES	15	At second precision, how far before the previous run's start the next run begins reading, 0 to 10080 minutes (a week). Ignored at day precision and on a full load
PAGE_SIZE	5000	Records per request, 1 to 20000; a value above 20000 is refused before anything is read. When SAP hands a request out in smaller slices, the flow follows them to the end. Tested sizes below
DELIVERY_SCHEDULE	every day at 02:00 UTC	When a run starts on its own; set with the scheduler control on the Configure screen. A deploy starts no run

Parameter	Default	Meaning
DELIVERY_ADDRESS	/keboola/deliver	The flow's own endpoint, reached as <code>https://<your tenant>/http/keboola/deliver</code> . Every copy of the flow needs an address of its own
SENDER_ROLE	ESBMessaging.send	The role a caller must hold to start a run through the endpoint (§1). Runs on the schedule need no role
S4_SAP_CLIENT	empty	Client number on a multi-client on-premise system; sent as <code>sap-client</code>
HTTP_TIMEOUT_MS	120000	Longest one call may take, a page from SAP or a call to Keboola, from 1000 to 3600000; longer counts as a passing failure and the run is retried
KEB00LA_STACK_URL	—	Your Keboola region address, for example <code>https://connection.eu-central-1.keboola.com</code>
KEB00LA_TABLE_ID	—	The table's identifier as Keboola shows it, for example <code>in.c-sap.business_partner</code> . The bucket must exist; the flow creates the table
KEB00LA_CREDENTIAL_ALIAS	KEB00LA_STORAGE	Security material holding the Storage token: <i>User Credentials</i> , any user name, the token as the password
KEB00LA_IMPORTER_URL	empty	Only to override the address looked up from the region
PRIMARY_KEY	BusinessPartner	Columns identifying a record uniquely, separated by commas; required for incremental loading
DELIVERY_QUEUE_NAME	KeboolaDelivery	Queue holding runs. Every copy of the flow needs a queue of its own
DELIVERY_RETRY_INTERVAL_MIN	3	Wait before the first retry of a run that failed for a passing reason; doubles each time
DELIVERY_MAX_RETRY_INTERVAL_MIN	60	The longest wait between retries
AIR_KEY, AIR_HEADER_NAME	empty	SAP's integration key, once issued (§8)

The shipped values read business partners. `SERVICE_PATH` , `ENTITY_SET` , `DELTA_FIELD` and `PRIMARY_KEY` ship as one example that works together. When you read another collection, change all four: the shipped key and change fields belong to business partners.

The schedule and the manual start

`DELIVERY_SCHEDULE` is set on the Configure screen with SAP's scheduler control and ships as every day at 02:00 UTC. Deploying the flow starts no run, whatever the schedule says, so a flow can be deployed, configured and deployed again without delivering anything. A scheduled run reads as of its own start time.

To run a delivery without waiting for the schedule, `POST` to the flow's address. It answers `202` with the envelope of the run it started, `messageId` first. That `messageId` is made by the flow, because the run itself is processed later from the queue: it is the run's *Application Message ID* in **Monitor** → **Message Search** and the log field `KeboolaMessageId` . A refused start answers `500` with `CONFIG_ERROR` and the setting named, or `405` for a wrong verb, and the body's `messageId` is then the run's id in Message Processing Logs. The endpoint asks for no CSRF token.

The settings are checked before anything is queued. A start is refused when `SERVICE_PATH` or `ENTITY_SET` is empty or holds a question mark or a blank, when `HTTP_TIMEOUT_MS` is outside its range, when `ODATA_SELECT` leaves out a column of `PRIMARY_KEY` , when incremental loading has no `PRIMARY_KEY` , when an address does not start with `https://` ([\\$2](#)), when a required setting is empty, when `LOAD_MODE` or `PAGE_SIZE` is outside its list or range, and, for incremental loading, when `DELTA_FIELD_TYPE` , `DELTA_PRECISION` or `DELTA_OVERLAP_MINUTES` is outside its list or range, or `DELTA_FIELD` is empty or not a comma-separated list of field names.

On the schedule there is nobody to answer. A start that is refused there is a run with the status `FAILED` and the same sentence in its error text. Next to every scheduled run the platform leaves a second entry with the status `DISCARDED` and no sender: it is the duplicate start on the other worker node, suppressed by the platform, and not a failure.

What is retried, and what is not

What happened	The run	Retried?
SAP answered 5xx, 408 or 429, or did not answer within <code>HTTP_TIMEOUT_MS</code> , or the line dropped	goes back on the queue, log entry <code>RETRY</code>	yes, by the platform, at the two intervals, until it succeeds or the message expires
Keboola's service index or importer answered 5xx, or did not answer	same	same
SAP answered a 4xx (403 no arrangement, 404 wrong path, 400 bad filter)	ends Escalated with the custom status <code>UPSTREAM_FAILED</code> , SAP's text attached, watermark not moved	no — fix the cause; the next run reads the same window again
SAP answered with something that is not a list of records: a web page, XML, JSON of another shape, or nothing	ends <code>Escalated</code> , <code>UPSTREAM_FAILED</code>	no — <code>S4_HOSTNAME</code> probably names a sign-in page or a web front end, not the API address of the system

What happened	The run	Retried?
SAP returned the same records for two requests in a row: the service does not apply \$skip, or PRIMARY_KEY does not identify one record (records that carry no address of their own, as in OData V4, are compared by their PRIMARY_KEY values)	ends Escalated, UPSTREAM_FAILED	no — check PRIMARY_KEY; if it is right, set PAGE_SIZE above the number of records, or read a service that pages
Keboola answered 401/403/400 — to the service index (the token), to the table check or to the import — or the table could not be created for a reason that will not change	ends Escalated, KEB00LA_FAILED	no
Keboola answered with a redirect	ends Escalated, KEB00LA_FAILED; the Storage token is never sent after a redirect	no — correct KEB00LA_STACK_URL
The table exists in Keboola with another primary key than PRIMARY_KEY	ends Escalated, KEB00LA_FAILED with both keys named; nothing is imported	no — set the same key in both places
A column name of the data, flattened, is longer than 64 characters	ends Escalated, KEB00LA_FAILED naming the column; nothing is imported	no — leave the field out with ODATA_SELECT
A setting is missing or wrong	the start is refused: 500 CONFIG_ERROR on the manual endpoint, a FAILED run on the schedule; inside a run, ends Escalated, CONFIG_ERROR	no
The run needs more pages than the flow reads in one run	ends Escalated, CONFIG_ERROR, watermark not moved	no — raise PAGE_SIZE or narrow ODATA_FILTER
The rows were delivered, but the point reached could not be saved	ends DELIVERED_WATERMARK_FAILED, the reason in the attachment WatermarkError	no — the data is in Keboola; the next run reads from the previous point again, and the primary key turns the repeats into updates
The sign-in to SAP could not be prepared: the security material named in S4_CREDENTIAL_ALIAS is missing, or of another type than the sign-in method needs	goes back on the queue, log entry RETRY	yes, until you correct the security material; the run then completes at its next retry

One repeat is not a failure. When SAP answered a request with more records than were asked for and the next request brings the first records again, SAP had handed out everything already: the run ends **DELIVERED** without importing the repeat, and its log says `page <n>`: the records of the first request again, not delivered: SAP S/4HANA had already handed out every record.

A page SAP itself cannot serve — a **500** from the service for a page that holds certain records, which some views produce at full width — counts as a passing failure: the run is retried with growing waits until you narrow **ODATA_SELECT** or **ODATA_FILTER**, or delete the waiting message. Such a run is visible in **RETRY** with SAP's sentence in **UpstreamError**.

Every retry is a run of the worker and at least one billed call to SAP, which is why a fault that would answer the same way every hour is not retried. The last row is the one exception: the platform reports that fault before the flow can judge it. Correct the security material, or delete the waiting message under **Monitor** → **Manage**

Message Queues.

Sizes and run times

The flow was tested with full loads of 190 134 journal entry items of 149 columns — about 5.7 KB of JSON a row, the widest record on the test system — at the three page sizes:

PAGE SIZE	Pages	Run time	Rows per second	Keboola import per page	SAP answer and conversion per page
1000	191	65 minutes	49	about 10 seconds	about 11 seconds
5000	39	17 minutes	184	about 10 seconds	about 16 seconds (29 MB a page)
20000	10	8 minutes	415	about 10 seconds	about 36 seconds (115 MB a page)

What the numbers say:

- **The cost of a load is the number of pages, not the bytes.** Every page is one Keboola import, and an import takes about ten seconds whatever the page holds. Small pages are paid for in imports: the same rows took four times longer at 1000 than at 5000.
- **5000, the shipped page size, is the recommendation** for rows of a few kilobytes. **20000 is the ceiling**, and it worked: a 115 MB page was held in memory as text, as parsed records and as the rows sent to Keboola without trouble on a tenant of SAP's standard sizing, and the other flows kept answering meanwhile. Use it for narrow rows; a wide record at 20000 means pages above 100 MB, which leave little room for a second large flow on the same node. 1000 is right only when SAP is slow per row or the record is very wide.
- **One run must finish within four hours**, the time the queue keeps a run locked to the worker that took it (the sender's Lock Timeout, shipped at 240 minutes); a run that outlives the lock can be handed out again. At 5000 rows a page and about 26 seconds a page that is about **550 pages, 2.7 million rows of this width**; at 1000 rows a page about 720 pages; at 20000 rows a page about 310 pages, six million rows. Narrower rows go faster.
- **Larger loads do not belong in one delivery run.** Read them through the pull path — the connector and Keboola's component, which page at their own pace — or split them over copies of the flow, each with an `ODATA_FILTER` of its own (below).
- Throughput is a floor for wide rows and a ceiling for nothing: the business partner example of five columns delivered 315 rows in four pages of 100 in about a minute, which is the per-page cost again.

One run per table at a time. Two runs writing the same table at once — a schedule and a manual `POST`, say — wait for each other at Keboola's importer, and each page takes longer. The shipped `HTTP_TIMEOUT_MS` of `120000` allows for that. With a lower value both runs go to `RETRY` and their retries fall due together again. Start the second run when the first shows `DELIVERED`; the flow's own concurrency setting does not keep two runs apart.

More than one table

One flow delivers one collection into one table. For another table, or to split a large load, copy the flow within the package and give the copy three things of its own, all on its Configure screen:

1. **An address**, in `DELIVERY_ADDRESS`, for example `/keboola/deliver/products`. Two flows cannot share an address; the copy deploys next to the original and answers on its own.
2. **A queue**, in `DELIVERY_QUEUE_NAME`. Copies that share a queue carry out each other's runs. Every queue counts against the tenant's queue quota; **Monitor** → **Manage Message Queues** shows what is in use.
3. **A table**, in `KEBOOLA_TABLE_ID`.

Two deliveries must not write into the same table. A full load of one replaces what the other delivered, and the point reached by incremental loading is kept per table, so each would skip what the other has yet to read.

Full loads

A full load replaces the table: the first page that carries records is imported as a full import, which clears what was there, and every later page is appended. That is page 1, unless SAP begins with an empty answer and a link to more. A full load is therefore **not atomic** — between the first and the last page a reader sees a partial table, for the minutes the run takes (above). If something reads the table continuously, use incremental loading with a primary key instead, where every page is an update.

A full load that reads no records leaves the table as it was. One rare case: if SAP answers a retried run differently from its first attempt, with records where the first attempt found none, the full load can add to the table instead of replacing it. The next full load puts it right.

Incremental loading

With `LOAD_MODE=incremental`, each run reads the records whose `DELTA_FIELD` lies between the start of the previous **successful** run and the start of this one — both ends inclusive, at the precision you set. At the precision `day` the period opens one day earlier still. Most SAP change fields carry only a *date*, and it is the date of the SAP system's own time zone, while the flow keeps its times in universal time: without the extra day a record changed late on the day of the previous run could be missed for good. At the precision `second` the period opens `DELTA_OVERLAP_MINUTES` earlier, 15 minutes as shipped: SAP writes a change a little after the moment it carries as its change time, and without the overlap a change committed late would fall between two runs. Reading the overlap again costs a few updates through the primary key; skipping it would lose rows. Set `DELTA_PRECISION=second` only for a field that carries a time in universal time, and `DELTA_FIELD_TYPE` to how the field is typed in the service — the flow writes the condition accordingly for both OData versions.

Name the creation field as well. Many SAP services leave the change field empty until a record is changed for the first time, so a condition on that field alone never reads a new record. `DELTA_FIELD` therefore takes several fields, separated by commas, and a record is read when any of them lies in the period. All of them must be of the type named in `DELTA_FIELD_TYPE`.

Deletions are not carried over. A record deleted in SAP has no change date to be read by, so it stays in the Keboola table. A full load, which replaces the table, is the way to drop it.

The period each run used is in monitoring as `DeltaWindow`, the overlap as `DeltaOverlap`, and the first query as `FirstPageQuery`. The platform keeps the first 200 characters of each.

The point reached is written only after every page has landed, so a run that fails half way re-reads its whole window next time rather than skipping the rows it never delivered. The first incremental run has nothing stored yet and reads everything up to its own start. The point reached is kept in the tenant's data store

`KeboolaDeliveryState` under the table's identifier (**Monitor** → **Manage Stores** → **Data Stores**); delete the entry, and the next run reads everything. A point that cannot be read — a damaged entry, say — is not silent: the run reads its full window, both of its entries in the monitor carry `WatermarkRead = failed, full window`, and the reason is in the log. A point that cannot be written after the rows landed ends the run `DELIVERED_WATERMARK_FAILED` with the reason in the attachment `WatermarkError`.

Incremental loading **requires** `PRIMARY_KEY`: the flow refuses to start without one.

Set a primary key

With `PRIMARY_KEY` set, delivering the same record again updates it, and the records are read in key order so that consecutive pages are consecutive slices of one ordering. Without one, a redelivered record is added again, and a full load pages in whatever order SAP chooses: the run says so in monitoring as `PagingUnsorted = true`. Set the key whenever you can — and always for incremental loading.

Some services carry a key that cannot be sorted by (`sap:sortable="false"` in `$metadata`, the generated key of the journal entry items among them): SAP refuses the `$orderby`, and the run ends Escalated with SAP's sentence. Such a service is read without a key, and split into copies of the flow by `ODATA_FILTER` — one fiscal period each, for instance — rather than paged by key. An aggregating view can also answer fewer rows than its `$count` when `ODATA_SELECT` names fewer columns than the view has; the rows delivered are the view's own count at that selection.

The key is given to the table when the flow creates it. A table that exists already keeps the key it has in Keboola; when `PRIMARY_KEY` is set and differs from it, the run ends `KEB00LA_FAILED` and names both.

The flow also refuses to import the same page of the same run twice, so a redelivered run does not re-import the pages that already landed. One case escapes it: a page that Keboola finished importing only after the flow had stopped waiting for the answer counts as not delivered, and the retry imports it again. With a primary key its records replace themselves. Without one they are in the table twice until the next full load.

The primary key is the second line of defence, and the one that also covers a record changing between two different runs. One residual case remains, and it is inherent to offset paging: rows inserted in SAP between two attempts of the *same* run can shift the page boundaries, so a page skipped as already delivered may not hold exactly the rows it held the first time. The next run's inclusive window reads them.

What the flow does about a missing table

The service that receives the data cannot create tables. On the first page that carries records the flow asks whether the target table exists; if it does not, it creates it from the columns of the data being delivered, with `PRIMARY_KEY` as the key, and waits for the creation to finish before importing. **The table is a typed table, and every column is created as `STRING`**: the flow cannot invent a business meaning SAP does not declare, and a wrong type is far more expensive to undo than a conversion later. The columns stand in the order of `ODATA_SELECT`, and in the order of SAP's answer when that is empty. A column name — a flattened structure can make a long one — must fit Keboola's limit of 64 characters; a longer one ends the run before anything is imported, naming the column.

7. When something goes wrong

Every failure is visible in **Monitor** → **Message Processing Logs**, filterable by status without opening a single message. Two columns matter: the platform's **status** — *Completed, Retry, Escalated, Failed* — and the package's **custom status** below.

Custom status	Meaning	What to do
OK	A pull request or a catalogue call was answered	—
QUEUED	A delivery run was put on the queue	—
DELIVERED	Every page of a delivery run landed	—
DELIVERED_WATERMARK_FAILED	Every page landed, but the point reached could not be saved	Read the attachment <code>WatermarkError</code> ; the next run re-reads from the previous point, which the primary key makes harmless
RETRY	A delivery run failed for a reason that may pass and is waiting on the queue	Nothing, unless it keeps happening: Monitor → Manage Message Queues shows the retry count and the next retry. A run whose sign-in to SAP could not be prepared waits there until the security material is corrected (§6)
UPSTREAM_FAILED	SAP refused the request and said why. In a delivery run (status <i>Escalated</i>): SAP refused a page, or answered with something that is not a list of records, and the run ended without a retry	Read the fields <code>UpstreamStatus</code> , <code>UpstreamErrorCode</code> and <code>UpstreamError</code> : SAP's own code and sentence. A delivery run has no <code>UpstreamErrorCode</code> , and its <code>UpstreamError</code> is the first 200 characters of SAP's answer. For a delivery run read the attachment <code>ErrorDetails</code> as well; the next run repeats the window. Usually a missing communication arrangement
UPSTREAM_UNREACHABLE, UPSTREAM_TIMEOUT	The connection could not be made, or SAP did not answer in time	Check <code>S4_HOSTNAME</code> , the Cloud Connector, and network reachability; for a timeout, SAP's own health and <code>HTTP_TIMEOUT_MS</code> . Keboola retries these on its own
UPSTREAM_ERROR	The catalogue could not read its list: SAP refused the sign-in, answered with an error, or grants no arrangement for the catalogue's own API. The message says which	For a refused sign-in check the security material against <code>S4_AUTH_METHOD</code> , and that the user is not locked in SAP; under OAuth 2.0 on an on-premise system, the client's scope (§2); on S/4HANA Cloud, <code>SAP_COM_0A07</code>
PATH_NOT_ALLOWED	The connector was asked for a path outside <code>CONNECTOR_PATH_PREFIXES</code>	Correct the service path in Keboola, or add the prefix (§4)
KEB00LA_FAILED	Keboola refused the import or the table for a reason that will not change (status <i>Escalated</i>)	Read the fields <code>KeboolaStatus</code> and <code>KeboolaError</code> (the first 200 characters of Keboola's answer) and the attachment <code>ErrorDetails</code> : token, bucket rights, table, primary key, a column name over 64 characters

Custom status	Meaning	What to do
CONFIG_ERROR	This flow is misconfigured	The message names the parameter. When it names S4_CREDENTIAL_ALIAS, the security material is missing or of another type than S4_AUTH_METHOD needs; the field ErrorLocation says where inside the platform the sign-in failed
CONNECTOR_ERROR, CATALOG_ERROR	The platform itself failed inside the connector or the catalogue, outside the sign-in; or, for the catalogue, something no other status describes	The message and the field ErrorLocation name the place; the attachment ErrorDetails the words. This is what to send to support
METHOD_NOT_ALLOWED	The caller's request was refused	The body says why
CATALOG_EMPTY	The catalogue found nothing	Assign the communication user to an arrangement
DELIVERY_NOT_STARTED	A manually started delivery could not be queued	The body says why
FAILED	A scheduled delivery was refused at its start	The error text of the run names the setting (§6)
DISCARDED	Next to every scheduled delivery run, the duplicate start the platform suppressed on the other worker node	Nothing; it is not a failure

A final failure of a delivery run is Escalated, with its custom status kept: the monitor shows *Escalated* and `KEB00LA_FAILED`, `UPSTREAM_FAILED` or `CONFIG_ERROR` side by side, the run's fields and attachments are all there, and the queue holds nothing for it. The platform's own error column of such a run is empty, because no exception was thrown; the reason is in the attachment `ErrorDetails` and the `KeboolaError` or `UpstreamError` field. The entry that queued the run stays *Completed* with `QUEUED`.

Alerting

Nothing in this package sends an alert; your tenant's monitoring does. What to watch:

- **Status Escalated** in Message Processing Logs — every delivery run whose data did not arrive. In the API of the tenant that is `MessageProcessingLogs?$filter=Status eq 'ESCALATED'; add and IntegrationFlowName eq 'DeliverBusinessDataFromSAPS4HANAToKeboola'` for one flow, or `CustomStatus eq 'KEB00LA_FAILED'` and the other custom statuses above for one cause.
- **Status Retry** that lasts: a delivery run waiting in the queue for longer than a few retry intervals.
- **The custom status DELIVERED_WATERMARK_FAILED** on a *Completed* run: the data arrived and the next window will be read twice.
- For the pull flows, every refused read is answered to Keboola in the same second and the Keboola job fails visibly; Keboola's own job notifications are the alert, and the run's custom status is there for the search.

SAP's central tool for this is **SAP Cloud ALM** (*Integration and Exception Monitoring*), which reads this tenant's message processing logs, raises alerts on failed and escalated messages and notifies by e-mail, chat or ticket; SAP Focused Run does the same for service providers. A small integration flow of your own on a timer that

reads the query above and posts to the **SAP Alert Notification service** or to an e-mail is the lightweight way. Entries with the status `DISCARDED` next to scheduled runs are normal and should not be alerted on.

Where a 401 or 403 comes from

A 401 or 403 in a Keboola job has three possible origins. When the run exists in the monitor with the status `UPSTREAM_FAILED`, SAP refused the user of the security material. When the run exists with `PATH_NOT_ALLOWED`, the service path is outside the connector's prefixes. When no run exists, your tenant refused Keboola's own key or role (§1).

Behind an SAP Cloud Connector a problem with the tunnel is answered by SAP's platform, not by SAP S/4HANA and not by this package, and the pull connector passes the answer on as it is:

Answer	SAP's sentence begins	Cause
403	Access denied to resource ... on system ...	The path is not among the mapping's resources
503, within a second	There is no SAP Cloud Connector (SCC) connected to your subaccount ... and SCC location ID ...	No Cloud Connector is paired under that Location ID: check <code>S4_CC_LOCATION_ID</code> and the pairing
503, after about ten seconds	Timed out waiting for tunnel to open ...	The Cloud Connector is paired but cannot open the data tunnel: §2, the third check
405	HTTPS proxying is not supported. Use HTTP instead ...	<code>S4_HOSTNAME</code> starts with <code>https://</code> ; on this route it is <code>http://</code> plus the virtual host

The catalogue answers `502 UPSTREAM_ERROR` instead and names the status of each catalogue request in its message. The delivery flow records the status and SAP's sentence in the fields `UpstreamStatus` and `UpstreamError`: a 503 sends the run to `RETRY`, a 403 or 405 ends it Escalated with `UPSTREAM_FAILED`. A run that waits in `RETRY` because of the tunnel is retried or deleted under **Monitor** → **Manage Message Queues** once the cause is fixed.

What the monitor holds

Searchable fields on every connector run: `HttpPath`, `UpstreamStatus`, `UpstreamOutcome`. When SAP refused the request, also `UpstreamErrorCode` and `UpstreamError` — SAP's own code and sentence; when the platform failed, `ErrorLocation`. SAP's whole answer goes back to Keboola and is not kept in the log; what Keboola sent is in the log property `IncomingHeaders`, with credentials redacted. When the call could not be made at all, the attachment `ErrorDetails` holds the cause and the attachment `IncomingHeaders` the request. On every catalogue run: `CatalogSource`, `CatalogCount`, `CatalogHidden`, `CatalogView`, `CatalogUnlisted`, `CatalogUnlistedNames`, and — as a log property on the run's *Properties* tab rather than a field — the notes of the run in `CatalogDiagnostics`. For deliveries: `Pages`, `RowsDelivered`, `TableId`, `Watermark`, `DeltaWindow`, `DeltaOverlap`, `FirstPageQuery`, `WatermarkRead`, `PagingUnsorted`, `PageOutcome` and `TableState` per page; on a refused page `UpstreamStatus`, `UpstreamOutcome` and `UpstreamError`; on a refused import `KeboolaStatus` and `KeboolaError`. These are fields, not attachments, so they can be filtered on and read by API; the platform keeps the first 200 characters of a field. Runs also carry SAP's standard search identifiers, so **Monitor** → **Message Search** finds them by sender, receiver or message type — and a connector run by the path that was read, which is its *Application Message ID*.

A failed pull request is not stored anywhere else: it is answered to Keboola in the same second, the Keboola job fails visibly with the connector's error body, and Keboola's own notifications apply. The run stays in Message Processing Logs for the tenant's retention period, findable by the `messageId` in that body.

The dead-letter queue

A delivery run that fails for a reason that may pass is retried by the platform with growing intervals — `DELIVERY_RETRY_INTERVAL_MIN` doubling up to `DELIVERY_MAX_RETRY_INTERVAL_MIN` — without a retry count: it keeps being retried until it succeeds or the queued message expires, 90 days after it was queued. While it waits it is a failed message in **Monitor** → **Manage Message Queues**, with its retry count and next retry time, and that is where you retry or delete it after fixing the cause. The platform's own dead-letter queue takes only a message whose processing crashed the runtime node, after two attempts. A run that fails for a reason that will not change is not queued for retry at all: it ends Escalated with `UPSTREAM_FAILED`, `KEB00LA_FAILED` or `CONFIG_ERROR` in Message Processing Logs, and the next scheduled run is the retry.

Editions without JMS

The delivery flow needs message queues and cannot be used on an edition without them. On such an edition:

- Do not deploy the delivery flow; deploying it fails for want of a queue.
- The two pull flows work unchanged, and Keboola's own scheduling replaces the delivery flow's.
- The supported fallback for error handling is the pull topology itself: every failed read is answered to Keboola synchronously with a status and a body naming the run, retried by Keboola with exponential backoff where retrying can help, surfaced in Keboola's job log and notifications, and filterable in this tenant's Message Processing Logs by custom status. Nothing waits in a queue, so nothing can be lost from one.

8. Things worth knowing before they surprise you

Retries are billed. Every call to SAP from a flow is a separately billed message, and a retry is another one. One execution of a flow counts as one message however many calls to SAP it makes — a catalogue read with its arrangement pages and verification requests is one message, as is one delivered page — under the metering rule SAP publishes in SAP Note 2942344; the same note announces metering by size, which was not switched on at the time of writing. The pull connector makes one call per request and leaves retrying to Keboola, whose three retries are the ceiling. The delivery flow's queue is not metered, but every retried run calls SAP again, which is why a fault that will not change ends the run instead of being retried.

Data residency. The data moves from your SAP system to your Integration Suite tenant to your Keboola project. Those three can be in different regions; choose the Keboola region and the tenant region deliberately.

Date formats. Older SAP services return dates as `/Date(1777939200000)/` and times as `PT19H02M07S`. The delivery flow rewrites both: a date becomes `yyyy-MM-ddTHH:mm:ss` with no time zone suffix, because SAP's value carries none, and a time becomes `19:02:07`, fractions of a second kept. It writes numbers in plain notation, and it does not deliver the control information of OData V4 (`@odata.etag` and the like) as columns. The pull connector passes SAP's answer through untouched — it is a proxy, and rewriting a payload in transit would make it something else.

The address book is a snapshot. The value mapping carries the released OData services of SAP S/4HANA Cloud as of September 2026. SAP releases new ones with every release; a service the mapping does not know is tried at guessed addresses, and when none verifies it is named as unlisted and you add the row (§3). The arrangements themselves never go stale, which is why they are the source and the mapping only the address book.

Later versions of a service. SAP publishes some services in a second version. The mapping lists a further version under `<name>_<version>`, and the catalogue looks an arrangement's service up that way first, so each version keeps its own address. Keboola's component leaves a version marker (`;v=0002`) out of an address, so a V2 service is always read in the version its plain address answers.

Paging depends on SAP's answers. The delivery flow asks for `PAGE_SIZE` records at a time and follows SAP's links to the rest. An answer with fewer records and no link is taken as the end, so a service that cuts its answers short without a link loses the rest unnoticed; no client can see that. A service that ignores `$skip` answers every request with the same records; the flow notices on the second request and ends the run Escalated (§6), unless SAP had handed out everything in one answer. One that ignores `$skip` and holds exactly `PAGE_SIZE` records ends that way too, because nobody can tell whether more exist. Records are told apart by their address, or without one by their `PRIMARY_KEY` values; with neither, such a service is read until the flow's page limit. For an OData V4 service, set `PRIMARY_KEY` to its key.

The integration key. SAP's partner program registers every integration that calls SAP applications in its Application Interface Register, and assigns the integration's publisher a key for it once the integration has passed SAP's partner solution review. Keboola, as the publisher of this package, receives that key and the name of the request header it travels in from SAP, and passes both on with this document; until then both fields stay empty and nothing is sent. Put the key in `AIR_KEY` and the header name in `AIR_HEADER_NAME` on each flow. The key identifies the integration to SAP and grants nothing — the sign-in to SAP is still the security material — which is why it is a plain Configure value, visible on the screen and in a trace like any other, rather than a secret. It goes to SAP only; it is not returned to the caller.

9. Rotating credentials

Four things sign in somewhere, and each is replaced without touching a flow's design.

Credential	Where it lives	How to rotate
The SAP communication user's password or client secret	The security material named in S4_CREDENTIAL_ALIAS	Set the new secret on the SAP side, then Monitor → Manage Security → Security Material → the entry → Edit , enter the new value and deploy the material; then redeploy every flow that uses it, so that no running copy keeps working with a sign-in it prepared earlier. Do it in that order, in one sitting: between SAP's change and the material's, pull reads answer 401 with UPSTREAM_FAILED and a delivery run ends Escalated
The client certificate	The key pair named in S4_PRIVATE_KEY_ALIAS	Add the renewed key pair to the keystore — under the same alias, or a new alias entered in S4_PRIVATE_KEY_ALIAS — map its public certificate to the communication user on the SAP system, and redeploy the flows
The Keboola Storage token	The security material named in KEB00LA_CREDENTIAL_ALIAS (KEB00LA_STORAGE)	Create the new token in Keboola, edit the material's password, deploy it, redeploy the delivery flow; only then revoke the old token. A delivery run that meets a revoked token ends Escalated with KEB00LA_FAILED and KeboolaStatus 401
The service key Keboola calls your tenant with	The service instance in the BTP cockpit, and the component's configuration in Keboola	Create a second key on the same instance, enter its client id and secret in the component's configuration, run one job, then delete the old key. No flow changes: the role, not the key, is what the flows ask for

A Cloud Connector's own certificates are the Cloud Connector's business and are renewed there.

10. The Keboola adapter

The adapter is for integration flows of your own. A flow that ends with it hands over the message body, comma separated values with a header line, and the adapter writes those rows into one Keboola Storage table and gives Keboola's answer back to the flow. It reads nothing from SAP and it does not page, queue or retry: whatever starts your flow and whatever feeds it rows is yours to build. When you want SAP data in Keboola without building anything, use the delivery flow (§6), which needs no adapter.

Install it before the flow that uses it. Deploy the *Keboola* integration adapter from the package's Artifacts tab first, then deploy your flow. After deploying a newer version of the adapter, deploy the flows that use it once more, so that none of them keeps working with the version it started with.

The credential. Create *User Credentials* under Monitor, Manage Security, Security Material: any user name, and the Keboola Storage token as the password. The token needs write access to the bucket of the target table. The token is read from there at every delivery and appears in no setting, log or error text.

Settings

Setting	Default	Meaning
Keboola Stack URL	—	The address of your Keboola stack, for example <code>https://connection.eu-central-1.keboola.com</code> . Must start with <code>https://</code>
Credential Name	KEB00LA_STORAGE	The name of the User Credentials above
Table	—	The full identifier of the table, for example <code>in.c-sap.business_partner</code> . The table must exist , with the columns of your header line; the adapter does not create it
Incremental Load	on	On: the rows are added. Off: the table's content is replaced by this delivery
Timeout (in ms)	60000	How long connecting and waiting for Keboola's answer may take. Must be greater than 0. Keboola answers when the import is finished; a file of a few rows typically takes 15 to 20 seconds

Every setting can be externalized.

What your flow gets back

On success the message body is Keboola's answer, for example

```
{"importedRowCount":315,"totalRowCount":315,"importedColumns":["id","name"],...}
```

, and the

adapter sets three message headers:

Header	Content
KeboolaHttpStatus	the status Keboola answered with
KeboolaTableId	the table written to
KeboolaIncremental	true or false

In a flow that answers an HTTPS caller, message headers travel back as response headers; remove the three in your flow if the caller should not see them. Towards Keboola the adapter sends one header of its own, `X-StorageApi-Token`, and passes none of the message's headers on.

When it fails

A failed delivery raises an exception in your flow, so an exception subprocess or the sender's retry decides what happens next. The text names the cause:

Text	What to do
... failed with status 401: Keboola did not accept the Storage token	The token in the credential is wrong, expired or revoked
... failed with status 404: Table ... not found	Create the table, or correct <i>Table</i>
... failed with status 4xx: <Keboola's sentence>	Keboola refused the file; the sentence is Keboola's own
Could not find credential for the given alias ...	<i>Credential Name</i> names nothing that is deployed
Keboola Stack URL must start with https://...	The address would send the token unencrypted; the adapter does not call it
Could not reach the Keboola service list at .../... answered with status ...	The stack address is wrong or Keboola does not answer. Nothing was sent; trying again is safe
The Keboola service list ... names no importer	The address is not a Keboola stack's connection address
Could not reach the Keboola importer at ...: Read timed out	Keboola did not answer within the timeout. The import may still have happened: check the table before sending the same rows again, or deliver with <i>Incremental Load</i> on into a table with a primary key, where a repeated row replaces itself
... the address redirects to ...; redirects are not followed	Something between your tenant and Keboola answers in Keboola's place. The adapter sends the token and the rows to the configured stack only

Size

The adapter sends one message as one file and holds that message in memory while it does. Keep a delivery to tens of megabytes and split larger data into several messages, as the delivery flow does with its pages.

11. Support and responsibilities

Keboola publishes and supports this package. Report a problem with the package to Keboola's support channel as named on the package's listing on the SAP Business Accelerator Hub, and send with it: the flow and its version, the custom status, the `messageId` of the run (from the error body, or the Message GUID in the monitor), the run's `ErrorDetails` attachment and fields, and what was configured — never the security material. For a Keboola job, the job id as well.

Who	Is responsible for
Keboola	The four artifacts and the adapter as delivered: their behaviour, this guide, new versions on the Hub, and the SAP OData component on the Keboola side
SAP	The Integration Suite platform the flows run on, the Cloud Connector, and SAP S/4HANA and its APIs, each under its own support
You	Your tenant and its sizing, the security material and keys, the communication user and its arrangements or authorizations, the Cloud Connector's mappings, the Configure values, the rows you add to the address book, and the Keboola project the data lands in

A flow you have edited in the flow editor is yours: it no longer receives the package's updates from the Hub, and support starts from the unedited version. Nothing in this guide needs the editor, so that case should not arise.

12. Change log

Version 1.1 answers Keboola's acceptance review of version 1.0; the items are numbered as in that review.

All flows

- The sign-in method to SAP S/4HANA is a Configure value, `S4_AUTH_METHOD`, with the credential name and the private key alias next to it; one build serves OAuth 2.0 client credentials, Basic and a client certificate (B1). Version 1.0 required an edit of each flow's receiver for anything but OAuth 2.0.
- The role a caller must hold is a Configure value, `SENDER_ROLE` (B4).
- Only the request headers an OData client needs travel to SAP, plus the integration key's header; the tenant's own message identifiers no longer do (header forwarding minor).
- The platform's own failures are no longer reported as sign-in failures: a sign-in failure is one the platform raised while preparing the sign-in, anything else is `CONNECTOR_ERROR` or `CATALOG_ERROR` with the place named (misleading error class minor).
- The integration key: where it comes from and that it is not a secret are stated (§8).

Query Business Data (the pull connector)

- `CONNECTOR_PATH_PREFIXES` limits what the connector forwards to the OData roots; anything else is `403 PATH_NOT_ALLOWED` (connector path scope minor).
- The tested maximum answer and SAP's own page cap are documented (§4, M7).

Query Available Services (the catalogue)

- Rebuilt. Every read of SAP goes through the flow's own connections, with the proxy type, Cloud Connector, sign-in method and certificate of the Configure screen; nothing is read from a script (B2).
- No probing of services with data reads, no parallel requests, no authorization check, no embedded service list, no cache; the only extra requests are at most four small service-document reads per unknown V4 service, bounded by `CATALOG_VERIFY_LIMIT` (B3, M9).
- The address book is a value mapping you maintain, *SAP S4HANA Service Addresses for Keboola* (M9).
- The support switches on the address (`?mode` , `?nocache` , `?filter`) are gone; `?debug=1` answers only with `CATALOG_DIAGNOSTICS=true` (B4).
- One call finishes within 50 seconds in every case; there is nothing cached, so nothing stale (M8).
- A wrong host is a `500 CONFIG_ERROR` before anything is listed, never an empty list (B3).
- Arrangement pages are followed to the end, up to 50 per set; the V4 catalogue's second response is kept apart from the V2 one; a `HEAD` request runs the read and answers without a body (catalogue paging minor).
- `CATALOG_CHECK_AUTHORIZATION` and `CATALOG_CACHE_TTL_SECONDS` are gone.

Deliver Business Data (the delivery flow)

- A final failure ends the run Escalated with its custom status kept, so the monitor shows it; §7 says how to alert on it (M4).
- `DELIVERY_ADDRESS` and `DELIVERY_SCHEDULE` are Configure values; a copy of the flow deploys next to the original from its Configure screen alone; a deploy starts no run (M5).
- Incremental loading at second precision overlaps the previous window by `DELTA_OVERLAP_MINUTES` ; a point reached that cannot be read or written is named in the monitor; a full load without a key is marked `PagingUnsorted` ; deletions are documented as not carried over (M6).
- `PAGE_SIZE` ships as 5000 and is capped at 20000; the tested sizes, run times and the longest safe run are in §6 (M7, M2, M3).
- Dates are written without a time zone suffix, times keep their fractions of a second, and a column name over 64 characters ends the run before any import (CSV conversion minor).
- The created table is documented as a typed table of `STRING` columns (table creation minor).
- The write path towards Keboola is unchanged: one import per page, a full load that is not atomic between its first and last page (M1, M2, M3 — documented with the tested numbers; a later version may move the write path into the adapter).

The Keboola adapter

- Unchanged. It is an optional building block for flows of your own; the delivery flow does not depend on it on purpose (M1).

This guide

- New: the Keboola side (§5), alerting (§7), credential rotation (§9), support and responsibilities (§11), this change log, the sources of the arrangement table (§3), the address book (§3).
- Corrected: the sign-in method is a Configure value, not an edit; the catalogue signs in like the other flows; the delivery schedule is a Configure value and a deploy starts no run.

Appendix A. Signing in to SAP S/4HANA, step by step

This appendix is about one connection: from the flows in your tenant to your SAP S/4HANA system. How Keboola signs in to your tenant is a different connection, described in §1.

Which method suits which system is the table in §2. The method is the Configure value `S4_AUTH_METHOD0D` of each flow; it ships as OAuth 2.0 Client Credentials. The security material named in `S4_CREDENTIAL_ALIAS` — or, for a certificate, the key pair named in `S4_PRIVATE_KEY_ALIAS` — must be of the type the method needs.

OAuth 2.0 Client Credentials

1. **Monitor** → **Manage Security** → **Security Material** → **Create** → **OAuth2 Client Credentials**.
2. Fill in:

Field	Value
Token Service URL	The token address of your SAP system. On-premise and private edition: <code>https://<host>/sap/bc/sec/oauth2/token?sap-client=<client></code> . S/4HANA Cloud: the token endpoint the communication arrangement shows
Client ID, Client Secret	The OAuth client registered on the SAP system, or the client id and secret the arrangement issued
Client Authentication	Send as request header, unless your SAP system expects it in the body

3. Deploy the security material.
4. On the Configure screen of each flow you use, leave `S4_AUTH_METHOD0D` at its shipped value and set `S4_CREDENTIAL_ALIAS` to the name of that security material.

Client Certificate

1. **Monitor** → **Manage Security** → **Keystore**. Add a key pair, or create one, and note its alias.
2. On the SAP system, assign the public certificate of that key pair to your communication user.
3. On the Configure screen of each flow you use, set `S4_AUTH_METHOD0D` to *Client Certificate* and `S4_PRIVATE_KEY_ALIAS` to the alias of the key pair. `S4_CREDENTIAL_ALIAS` is not used with this method.

This method cannot be used behind an SAP Cloud Connector (§2).

Basic

1. **Monitor** → **Manage Security** → **Security Material** → **Create** → **User Credentials**. Enter the communication user of the SAP system and its password.
2. Deploy the security material.
3. On the Configure screen of each flow you use, set `S4_AUTH_METHOD0D` to *Basic* and `S4_CREDENTIAL_ALIAS` to the name of that security material.

Switching the method

The three values are set once per flow, on its Configure screen, and the flow is deployed again. Every connection the flow has to SAP S/4HANA takes them — the catalogue's four as much as the other flows' one — so there is nothing to change inside a flow. A method chosen from the list that the platform cannot apply fails the deploy visibly, and the copy that was running before keeps answering until the value is corrected.

When the sign-in does not work

What you see	Cause
500 CONFIG_ERROR that names S4_CREDENTIAL_ALIAS and S4_AUTH_METHOD, from the pull connector or the catalogue	The security material is missing, or of another type than the method needs; the run's ErrorLocation field names the place inside the platform
A delivery run that stays in RETRY	The same fault in the delivery flow. The run completes at its next retry once the security material is corrected
401 from SAP with the status UPSTREAM_FAILED on the pull connector, 502 UPSTREAM_ERROR on the catalogue	SAP refused the user, the password or the token, or the user is locked in SAP
502 UPSTREAM_ERROR from the catalogue naming a 403 and a 500 from the Gateway catalogues, while the pull connector reads	OAuth 2.0 client credentials against an on-premise system whose OAuth client lacks the scope of the two Gateway catalogue services (§2)

SAP and other SAP products and services mentioned herein as well as their respective logos are trademarks or registered trademarks of SAP SE (or an SAP affiliate company) in Germany and other countries. All other product and service names mentioned are the trademarks of their respective companies.